Unveiling Secrets: Harnessing Frida, Ghidra, and Decompilers for Enhanced Analysis of Android and Apple Applications

Who am I?

- Senior Security Consultant
- Worked in IT Security for 6+ years
- Started on the blue team as a Network Security Analyst
- Started on the red team with Network and Wireless penetration testing then transisitoned to Web Application and Mobile

Tools Being Used in Talk

- Frida (Both)
- Text Editor (Using VSCode but doesn't matter)
- Jadx-gui (Android)
- Ghidra (Both)
- Hopper (Apple only)

Frida

- What is frida?
- Dynamic instrumentation toolkit / Tool that attacks memory
- Really exciting tool with wild abilities just waiting to be tapped into
- Injection scripts written in Javascript
- Can be used in a LOT of things
  - Android, Apple, MacOS, Linux, Windows, etc...

Text Editor

- I like sublime and Visual Studio Code
- GitHub CoPilot is also helpful for writing Frida scripts
- A little bit difficult to edit and save code without a means to edit and save the code

Jadx-gui

- Why use Jadx-gui?
  - Pros
    - Easy to use/navigate
    - Friendlier interface
    - Can create frida code easily
    - Open Source
  - Cons
    - Only can be used on Dex and Apk files (Android)
    - Doesn't decompile all the code

Jadx-gui Interface

- Nice and clean! Not that scary at all!

*Challenge 0x1 - jadx-gui

File View Navigation Tools Help

Challenge 0x1.apk
- Source code
  - android.support.v4
  - androidx
  - com
    - ad2001.frida0x1
      - MainActivity
      - R
    - google
  - kotlin
  - kotlinx.coroutines
  - org
- Resources
- APK signature
- Summary

MainActivity

```
package com.ad2001.frida0x1;

import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import java.util.Random;

/* loaded from: classes.dex */
26 public class MainActivity extends AppCompatActivity {
    TextView t1;

    /* JADX INFO: Access modifiers changed from: protected */
    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivity, android.app.Activity
27  public void onCreate(Bundle bundle) {
28      super.onCreate(bundle);
29      setContentView(R.layout.activity_main);
31      final EditText editText = (EditText) findViewById(R.id.editTextTextPassword);
33      this.t1 = (TextView) findViewById(R.id.textview1);
34      final int i = get_random();
36      ((Button) findViewById(R.id.button)).setOnClickListener(new View.OnClickListener() { // from class: com.ad2001.frida0x1.MainActivity.1
            @Override // android.view.View.OnClickListener
39          public void onClick(View view) {
40              String obj = editText.getText().toString();
43              if (TextUtils.isDigitsOnly(obj)) {
44                  MainActivity.this.check(i, Integer.parseInt(obj));
                } else {
48                  Toast.makeText(MainActivity.this.getApplicationContext(), "Enter a valid number !!", 1).show();
                }
            }
```

Ghidra

- Why use Ghidra?
  - Pros
    - Works with both IPAs and APKs
    - Has a lot of nice features such as code flow and function graph
    - Worth learning and spending time becoming proficient
  - Cons
    - Intimidating interface
    - Everything on the initial page is overwhelming at first

Showing what Ghidra looks like

- Scary at first, but the decompile page is great!

File Edit Analysis Graph Navigation Search Select Tools Window Help

Program Trees

- classes.dex
  - method_lookup
  - method_bytecode
  - header
  - data
  - classes
  - class_static_values
  - class_data
  - code_item
  - encoded_fields
  - encoded_methods
  - debug_info
  - handlers

Program Tree

Symbol Tree

- Imports
- Exports
- Functions
- Labels
- Classes
- Namespaces
  - android
  - CatchHandlers
  - com
    - ad2001
    - frida0x1

Filter:

Data Type Manager

- Data Types
  - BuiltInTypes
  - classes.dex

Listing: classes.dex

```
                    **************************************************************
                    * Class: Lcom/ad2001/frida0x1/MainActivity;                  *
                    * Class Access Flags:                                        *
                    *         ACC_PUBLIC                                         *
                    *                                                            *
                    * Superclass: Landroidx/appcompat/app/AppCompatActivity;     *
                    * Source File: MainActivity.java                             *
                    *                                                            *
                    * Method Signature: V( I                                     *
                    *         I                                                  *
                    *          )                                                 *
                    * Method Access Flags:                                       *
                    *                                                            *
                    * Method Register Size: 6                                    *
                    * Method Incoming Size: 3                                    *
                    * Method Outgoing Size: 3                                    *
                    * Method Debug Info Offset: 0x43ea93                         *
                    * Method ID Offset: 0xd6288                                  *
                    *                                                            *
                    **************************************************************
                    void __stdcall check(MainActivity * this, int p1, int p2)
        void            <VOID>          <RETURN>
        MainActivity *  ivo:4           this
        int             iv1:4           p1
        int             iv2:4           p2
        undefined4      v0:4            local_0
        undefined4      v1:4            local_1
        undefined4      v2:4            local_2
                    com::ad2001::frida0x1::MainActivity::check      XREF[3]:     Entry Point(*),
                                                                                 onClick:502c8b60(c)
502c8bdc da 04 04 02    mul_int_...  v4,v4,0x2
502c8be0 d8 04 04 04    add_int_...  v4,v4,0x4
502c8be4 12 10          const_4      local_0,0x1
502c8be6 33 54 4e 00    if_ne        v4,v5,LAB_502c8c82
502c8bea 6e 10 0c       invoke_v...  offset com::ad2001::frida0x1::MainActivity::ge...
         75 03 00
502c8bf0 0c 04          move_res...  v4
502c8bf2 1a 05 6d 66    const_st...  v5,offset strings::Yey_you_guessed_it_right
502c8bf6 71 30 ae       invoke_s...  offset android::widget::Toast::makeText,v4,v5,...
         11 54 00
502c8bfc 0c 04          move_res...  v4
```

Decompile: check - (classes.dex)

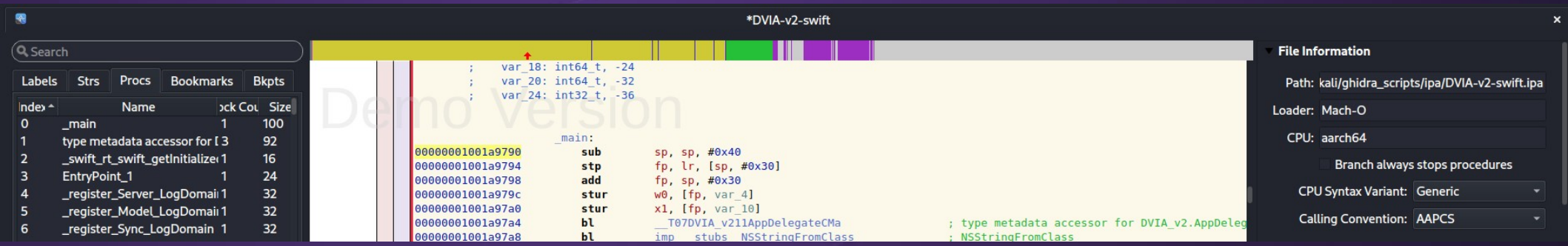
```
1
2  /* Class: Lcom/ad2001/frida0x1/MainActivity;
3     Class Access Flags:
4      ACC_PUBLIC
5
6     Superclass: Landroidx/appcompat/app/AppCompatActivity;
7     Source File: MainActivity.java
8
9     Method Signature: V( I
10     I
11      )
12    Method Access Flags:
13
14    Method Register Size: 6
15    Method Incoming Size: 3
16    Method Outgoing Size: 3
17    Method Debug Info Offset: 0x43ea93
18    Method ID Offset: 0xd6288
19     */
20
21 void check(MainActivity this,int p1,int p2)
22
23 {
24   char cVar1;
25   Context pCVar2;
26   Toast pTVar3;
27   String pSVar4;
28   char cVar5;
29   undefined ref;
30   StringBuilder ref_00;
31   int iVar6;
32   TextView ref_01;
33
34   if (p1 * 2 + 4 == p2) {
35     pCVar2 = this.getApplicationContext();
36     pTVar3 = Toast.makeText(pCVar2,"Yey you guessed it right",1);
37     pTVar3.show();
38     ref_00 = new StringBuilder();
39     for (iVar6 = 0; iVar6 < 0x14; iVar6 = iVar6 + 1) {
40       ref = "AMDYV{WVWT_CJJF_0s1}";
41       cVar1 = ref.charAt(iVar6);
42       if ((cVar1 < 'a') || ('z' < cVar1)) {
```

Hopper

- Why use Hopper?
  - Pros
    - Disassembler purpose built for Apple programs
    - Demo version is free
    - Less overwhleming that Ghidra
  - Cons
    - Only tool mentioned that is not open source
    - More than 30 minutes at a time and full functionality costs $99
    - Can't be used on Windows. Only Mac, Ubuntu 20.04 (64 bits), Fedora 31 (64 bits), and ArchLinux (64 bits)

How does Hopper look???

- Scary version (Assembly)



- Less scary version (Pseudo Code of Procedure)

Last important concept

- 2 ways we are going to interact with the apps
  - Directly calling them
  - Hooking
- Android:
  - Doesn't require
  - Calling directly creates a new class object
  - Overloading or hooking their implementation
- Apple
  - + [ClassName FunctionName]
  - - [ClassName FunctionName]

Easy option first!

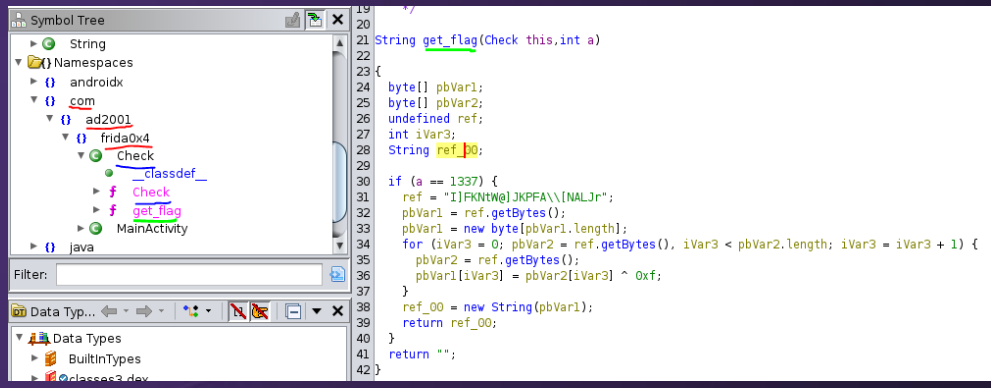
Android!

Calling a function and get the return

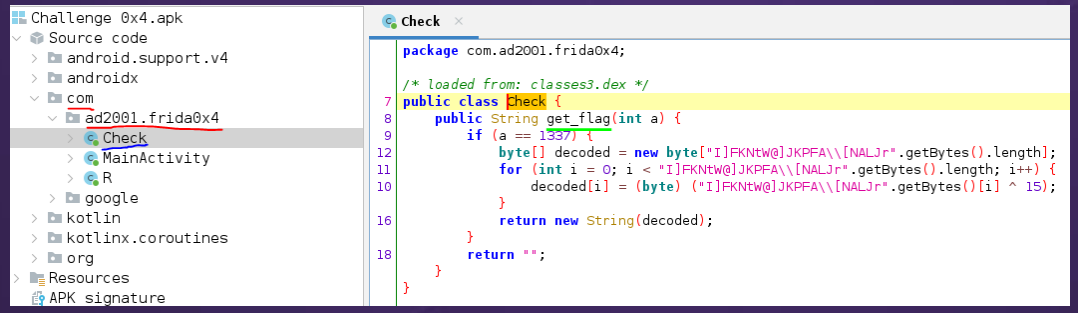
Legend ->

```
Red Lines:
    <package_name>
    com.ad2001.frida0x4

Blue Lines:
    <class>
    Check

Green Lines:
    <Method>
    get_flag(1337)
```

What to look for in Ghidra



What to look for in Jadx-gui



Code to get the returned string

```
console.log("\n[+] Started");
console.log("[*] " + Process.id);

if (Java.available) {
    console.log("[*] Java is available");
    Java.perform(function() {
        var check = Java.use("com.ad2001.frida0x4.Check");
        var check_obj = check.$new(); // Class Object
        var res = check_obj.get_flag(1337); // Calling the method
        console.log("[+] FLAG " + res);
    });
} else {
    console.log("[!] Java is not available");
}

console.log("[+] Done");
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL COMMENTS

```
Attaching...

[+] Started
[*] 13261
[*] Java is available
[+] FLAG FRIDA{XORED_INSTANCE}
[+] Done
```

Breakdown of code

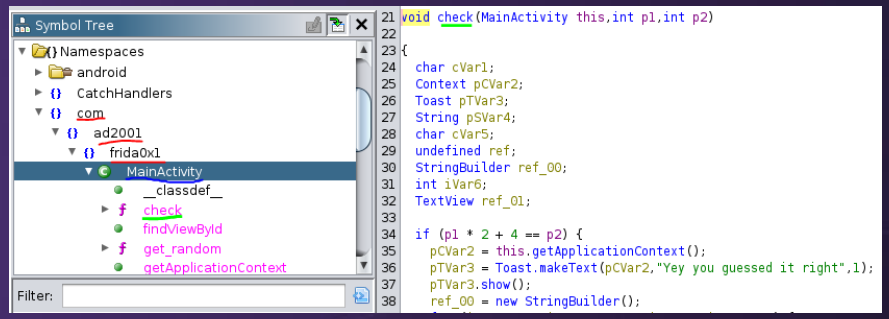
```
Java.perform(function() {

  var <class_reference> = Java.use("<package_name>.<class>");
  var <class_instance> = <class_reference>.$new(); // Class Object
  <class_instance>.<method>(); // Calling the method

})
```

App calls function and get the return value

Legend ->

```
Red Lines:
    <package_name>
    com.ad2001.frida0x1

Blue Lines:
    <class>
    MainActivity

Green Lines:
    <Method>
    check(a,b)
```

What to look for in Ghidra

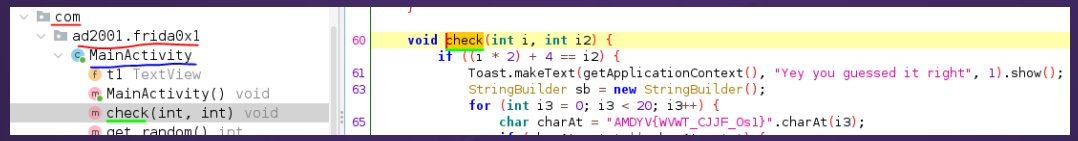


Code to get the returned string

```
console.log("\n[+] Started");
console.log("[*] " + Process.id);

if (Java.available) {
    console.log("[*] Java is available");
    Java.perform(function() {

        var a = Java.use("com.ad2001.frida0x1.MainActivity");
        console.log("Overloading check()");
        a.check.overload('int', 'int').implementation = function(a, b) {
            // The function takes two arguments; check(random, input)
            console.log("\nThe random number is " + a);
            console.log("The user input is " + b);
            var sol = a * 2 + 4;
            if (sol != b) {
            console.log(b + " is the wrong solution. Please try " + sol);
            }
            console.log("Solution is " + (a * 2 + 4));
            this.check(a, b); // Call the check() function with the correct arguments
        }
    });
} else {
    console.log("Java is not available");
}

console.log("[+] Done");
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   COMMENTS        python3 - ghidra_s

Attaching...

[+] Started
[*] 20281
[*] Java is available
Overloading check()
[+] Done
[SM A025U::PID::20281 ]->
The random number is 97
The user input is 198
Solution is 198
```

What to look for in Jadx-gui



Breakdown of code

```
var a = Java.use("<package_name>.<class>");
console.log("Overloading check()"); // Log to know where you are
a.<function>.overload(<Type of arg1>, <Type of arg2>).implementation = function(arg1, arg2) {
    // The function takes two arguments; check(random, input)
    console.log("\nThe random number is " + <arg1>);
    console.log("The user input is " + <arg2>);
    var sol = <arg1> * 2 + 4;
    if (sol != <arg2>) {
    console.log(<arg1> + " is the wrong solution. Please try " + sol);
    }
    console.log("Solution is " + (<arg1> * 2 + 4));
    this.check(<arg1>, <arg2>); // Call the check() function with the correct arguments
}
```

Taking the the next level

- Other methods like StringBuilder and Request
- Crypto libraries and other custom libraries that are app specific

```
var req = Java.use("okhttp3.Request");
req.url.overload().implementation = function(){
    var ret = req.url.overload().call(this);
    console.log("\n------url-----");
    console.log("[*] This object: " + this);
    if (ret != null)
        console.log("[*] ret toString: " + ret.toString());
    console.log("--------------");
    return ret;
}

var StringBuilder = Java.use('java.lang.StringBuilder');
StringBuilder.toString.implementation = function() {

    var res = this.toString();
    console.log("[!] Strings: " + res);
    return res;
};
```

Real world example

Code Example ->

```
var Log = Java.use("android.util.Log");
Log.d.overload("java.lang.String", "java.lang.String").implementation = function (a, b) {
    console.log("[*] Log.d: -" + a + "- -" + b + "-");
}

var req = Java.use("okhttp3.Request");
req.url.overload().implementation = function(){
    var ret = req.url.overload().call(this);
    console.log("\n------url-----");
    console.log("[*] This object: " + this);
    if (ret != null)
        console.log("[*] ret toString: " + ret.toString());
    console.log("--------------");
    return ret;
}

var StringBuilder = Java.use('java.lang.StringBuilder');
StringBuilder.toString.implementation = function() {

    var res = this.toString();
    console.log("[!] Strings: " + res);
    return res;
};
```

Output of example code

```
[+] Starting
[+] In it
[+] Everything is loaded! Go for it <3
[!] Strings: com.google.android.material.snackbar.Snackbar.SnackbarLayout
[!] Strings: com.google.android.material.snackbar.Snackbar.SnackbarLayout
[!] Strings: Q10001
[!] Strings: OkHttp ConnectionPool
[!] Strings: http://prod.allsafe.infosecadventures.io/api/v1/note/add?auth_token=YWxsc2FmZV9kZXZfYWRtaW5fdG9rZW4%3D&note=test
[*] Log.d: -ALLSAFE- -http://prod.allsafe.infosecadventures.io/api/v1/note/add?auth_token=YWxsc2FmZV9kZXZfYWRtaW5fdG9rZW4%3D&note=test-

------url-----
Error: Implementation for d expected return value compatible with int
    at ne (frida/node_modules/frida-java-bridge/lib/class-factory.js:674)
    at <anonymous> (frida/node_modules/frida-java-bridge/lib/class-factory.js:651)
[!] Strings: Request{method=GET, url=http://prod.allsafe.infosecadventures.io/api/v1/note/add?auth_token=YWxsc2FmZV9kZXZfYWRtaW5fdG9rZW4%3D&note=test}
[*] This object: Request{method=GET, url=http://prod.allsafe.infosecadventures.io/api/v1/note/add?auth_token=YWxsc2FmZV9kZXZfYWRtaW5fdG9rZW4%3D&note=test}
[*] ret toString: http://prod.allsafe.infosecadventures.io/api/v1/note/add?auth_token=YWxsc2FmZV9kZXZfYWRtaW5fdG9rZW4%3D&note=test
--------------
```

Now for the harder one!

Apple!

Differences between Android and Apple

- Harder to Reverse Engineer
- Order for attack is going to be different
- We can use frida to get methods and then we can look them up with ghidra/hopper
- Flow:
  - <GitHub Repo URL>/iOS/find-app-classes.js
  - Use frida-trace and see if it is called as you expect
  - <GitHub Repo URL>/iOS/find-specific-method.js
  - Use interceptor.attach

Apple Reverse Engineering

Flow Demonstration:

find-app-classes.js output

```
[+] DVIA_v2.JailbreakDetectionViewController
```

Frida-trace command

```
frida-trace -U -m "*[DVIA_v2.JailbreakDetectionViewController *]" -p 6540
```

Frida-trace output

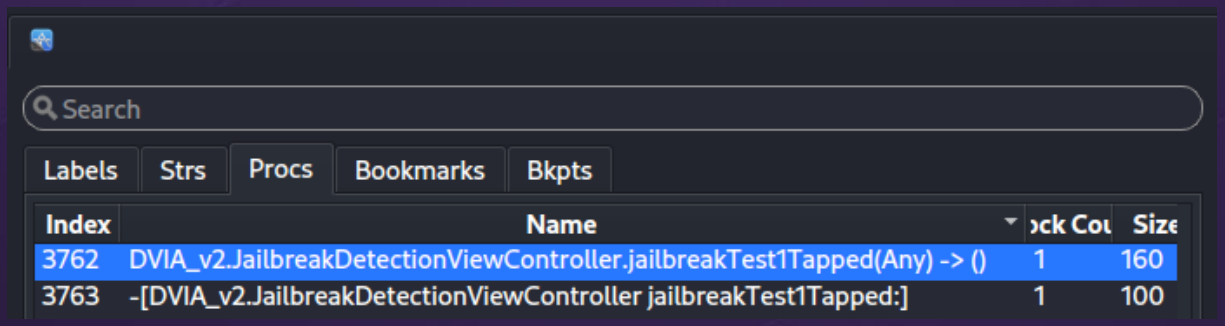
```
1777 ms  -[DVIA_v2.JailbreakDetectionViewController initWithCoder:0×103022000]
1806 ms  -[DVIA_v2.JailbreakDetectionViewController viewDidLoad]
1810 ms  -[DVIA_v2.JailbreakDetectionViewController viewWillAppear:0×0]
6399 ms  -[DVIA_v2.JailbreakDetectionViewController jailbreakTest1Tapped:0×102f58530]
```

find-specific-method.js output

```
[+] Class: DVIA_v2.JailbreakDetectionViewController
        [-] Method: - jailbreakTest1Tapped:
                [-] Arguments Type: pointer,pointer,pointer
                [-] Return Type: void
[*] Completed: Find Specific Method in All Classes
```

Method: - vs +

Called on vs Create instance



Looking up package / method type / method information on Hopper

Putting it together

Frida Script ->

```
function hook_specific_method_of_class(className, funcName)
{
    var hook = ObjC.classes[className][funcName];
    Interceptor.attach(hook.implementation, {
      onEnter: function(args) {
        // args[0] is self
        // args[1] is selector (SEL "sendMessageWithText:")
        // args[2] holds the first function argument, an NSString
        console.log("[*] Detected call to: " + className + " -> " + funcName);
        //For viewing and manipulating arguments
        console.log("\t[-] Value2: "+(ObjC.Object(args[2])).toString());
      },
      onLeave: function(retval){
        console.log("\t[-] retval: " + retval);
      }
    });
}

//Your class name  and function name here
hook_specific_method_of_class("DVIA_v2.JailbreakDetectionViewController", "- jailbreakTest1Tapped:")
```

Frida Output

```
[*] Detected call to: DVIA_v2.JailbreakDetectionViewController → - jailbreakTest1Tapped:
        [-] Value2: <UIButton: 0x143d4ead0; frame = (129 200; 117 33); opaque = NO; autoresize = RM+BM; layer = <CALayer: 0x2820b3220>>
        [-] retval: 0x143d45ee0
```

Ghidra

```
void __T07DVIA_v232JailbreakDetectionViewControllerC20jailbreakTest1TappedyypF
                (undefined8 param_1,undefined8 param_2,_NSZone *param_3)

{
  uint uVar1;
  long lVar2;
  SEL extraout_x1;
  ulong *unaff_x20;
  code *pcVar3;

  lVar2 = __T07DVIA_v213DVIAUtilitiesCMa();
  pcVar3 = *(code **)(lVar2 + 0x58);
  uVar1 = (**(code **)((*unaff_x20 & *(ulong *)PTR__swift_isaMask_1003a4708) + 0x80))();
  _objc_retain((ID)unaff_x20,extraout_x1,param_3);
  (*pcVar3)(uVar1 & 1,unaff_x20);
  ___swift_destroy_boxed_opaque_existential_0(param_1);
  return;
}
```

Hopper

Search

Labels | Strs | Procs | Bookmarks | Bkpts

Index	Name	ock Cou	Size
3762	DVIA_v2.JailbreakDetectionViewController.jailbreakTest1Tapped(Any) -> ()	1	160
3763	-[DVIA_v2.JailbreakDetectionViewController jailbreakTest1Tapped:]	1	100
3764	DVIA_v2.JailbreakDetectionViewController.jailbreakTest2Tapped(Any) -> ()	3	192
3765	type metadata accessor for __C.JailbreakDetection	3	88
3766	-[DVIA_v2.JailbreakDetectionViewController jailbreakTest2Tapped:]	1	100

```
/* @class _TtC7DVIA_v232JailbreakDetectionViewController */
-(int)jailbreakTest1Tapped:(int)arg2 {
    swift_unknownRetain(arg2);
    [arg0 retain];
    Swift._bridgeAnyObjectToAny();
    DVIA_v2.JailbreakDetectionViewController.jailbreakTest1Tapped(&stack[-64]);
    r0 = [arg0 release];
    return r0;
}
```

Real World Example

ClassName and Function ->

```
if (ObjC.available) {
    try {
        //Hooking fileExistsAtPath:
        Interceptor.attach(ObjC.classes.NSFileManager["- fileExistsAtPath:"].implementation, {
            onEnter(args) {
                // Use a marker to check onExit if we need to manipulate
                // the response.
                this.is_common_path = false;
                // Extract the path
                console.warn("fileExistsAtPath: " + new ObjC.Object(args[2]).toString());
                // console.log("fileExistsAtPath: " + new ObjC.Object(args[1]).toString());
                this.path = new ObjC.Object(args[2]).toString();
                // check if the looked up path is in the list of common_paths
                if (jailbreakPaths.indexOf(this.path) >= 0) {
                    // Mark this path as one that should have its response
                    // modified if needed.
                    this.is_common_path = true;
                }
            },
            onLeave(retval) {
                // stop if we dont care about the path
                if (!this.is_common_path) {
                    return;
                }

                // ignore failed lookups
                if (retval.isNull()) {
                    console.log(`fileExistsAtPath: try to check for ` + this.path + ' was failed');
                    return;
                }
                console.log(`fileExistsAtPath: check for ` + this.path + ` was successful with: ` + retval.toString() +
                retval.replace(new NativePointer(0x00));
            },
        });
```

Output of script

```
[iOS Device::PID::6399 ]-> fileExistsAtPath: /Applications/Cydia.app
fileExistsAtPath: check for /Applications/Cydia.app was successful with: 0x1, marking it as failed.
fileExistsAtPath: /Library/MobileSubstrate/MobileSubstrate.dylib
fileExistsAtPath: check for /Library/MobileSubstrate/MobileSubstrate.dylib was successful with: 0x1, marking it as failed.
fileExistsAtPath: /bin/bash
fileExistsAtPath: check for /bin/bash was successful with: 0x1, marking it as failed.
fileExistsAtPath: /usr/sbin/sshd
fileExistsAtPath: check for /usr/sbin/sshd was successful with: 0x1, marking it as failed.
fileExistsAtPath: /etc/apt
fileExistsAtPath: check for /etc/apt was successful with: 0x1, marking it as failed.
canOpenURL: check for cydia://package/com.example.package was successful with: 0x1, marking it as failed.
```

Why does this matter?

- Taking steps to understand an application deeper
- Information being returned from a function
- Being able to control information going into a function

Tutorials used in presentation

Android

- https://github.com/DERE-ad2001/Frida-Labs/tree/main/Frida%200x4
- https://github.com/DERE-ad2001/Frida-Labs/tree/main/Frida%200x1
- https://github.com/t0thkr1s/allsafe

Apple

- https://github.com/prateek147/DVIA-v2/tree/master

Resources to Learn More!

- Frida on Android
  - https://github.com/DERE-ad2001/Frida-Labs/tree/main
  - https://github.com/t0thkr1s/allsafe
- Frida on Apple
  - https://github.com/prateek147/DVIA-v2/tree/master
  - https://github.com/interference-security/frida-scripts/tree/master/iOS

References

- Android
  - https://udemy.com/course/hacking-and-pentesting-android-applications
- Apple
  - https://www.youtube.com/watch?v=mLDsIMXafP4
  - https://udemy.com/course/hacking-and-pentesting-ios-applications
  - https://udemy.com/course/the-complete-ios-pentesting-bug-bounty-course

Questions?